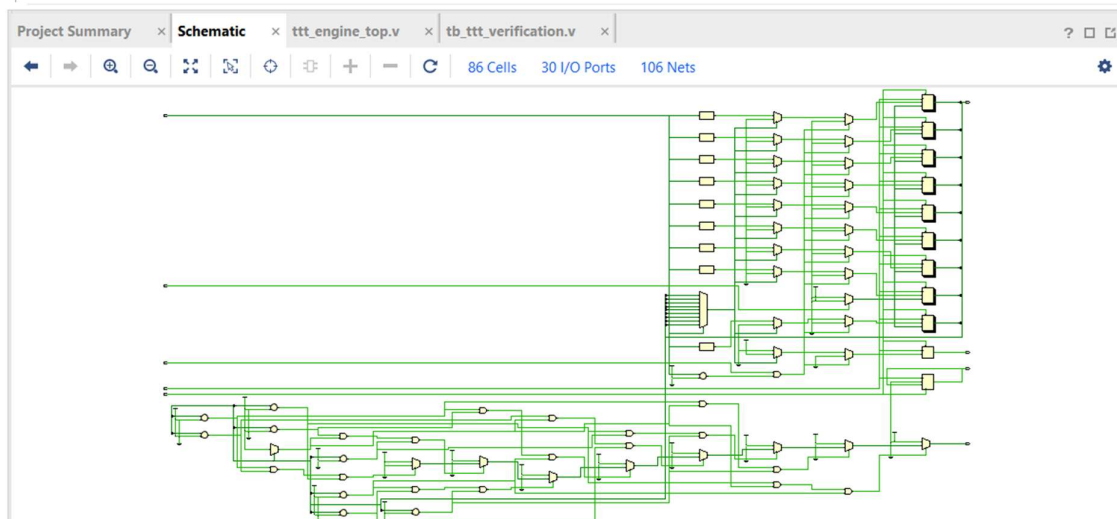
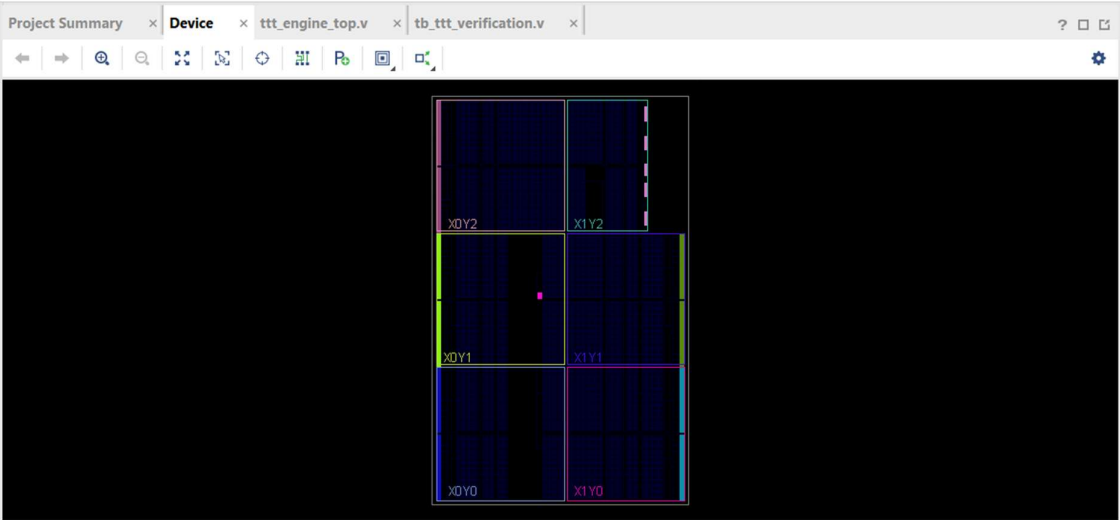


Results



```
source C:/Vivado_Projects/tictactoe_sim/tictactoe_sim.sim/sim_1/behav/xsim/xsim.dir/tb_ttt_verification_behav/webtalk/xsim_webtalk.tcl -notrace
INFO: [Common 17-206] Exiting Webtalk at Tue Sep 22 10:47:53 2026...
run_program: Time (s): cpu = 00:00:01; elapsed = 00:00:06 . Memory (MB): peak = 751.109 ; gain = 0.000
INFO: [USF-XSim-69] 'elaborate' step finished in '5' seconds
INFO: [USF-XSim-4] Xsim:Simulate design
INFO: [USF-XSim-61] Executing 'SIMULATE' step in 'C:/Vivado_Projects/tictactoe_sim/tictactoe_sim.sim/sim_1/behav/xsim'
INFO: [USF-XSim-98] *** Running xsim
    with args "tb_ttt_verification_behav -key {Behavioral:sim_1:Functional:tb_ttt_verification} -tclbatch {tb_ttt_verification.tcl} -log {simulate.log}"
INFO: [USF-XSim-8] Loading simulator feature
Vivado Simulator 2018.3
Time resolution is 1 ps
source tb_ttt_verification.tcl
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0 } {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#     send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Wave
#   }
# }
# run 1000ns
===== STARTING RTL ENGINE VERIFICATION MATRIX =====
[TC1 SUCCESS] Engine rejected collision step successfully.
[TC2 SUCCESS] Engine recognized combinational diagonal winner resolution.
===== RTL VERIFICATION SUITE COMPLETE =====
$finish called at time : 127 ns : File "C:/Vivado_Projects/tictactoe_sim/tictactoe_sim.srsrc/sim_1/new/tb_ttt_verification.v" Line 67
INFO: [USF-XSim-96] Xsim completed. Design snapshot 'tb_ttt_verification_behav' loaded.
INFO: [USF-XSim-97] Xsim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:04; elapsed = 00:00:12 . Memory (MB): peak = 751.109 ; gain = 0.000
```





Tcl Console Messages Log Reports Design Runs Timing x

Design Timing Summary

General Information

Timer Settings

Design Timing Summary

> Check Timing (93)

Intra-Clock Paths

Inter-Clock Paths

Other Path Groups

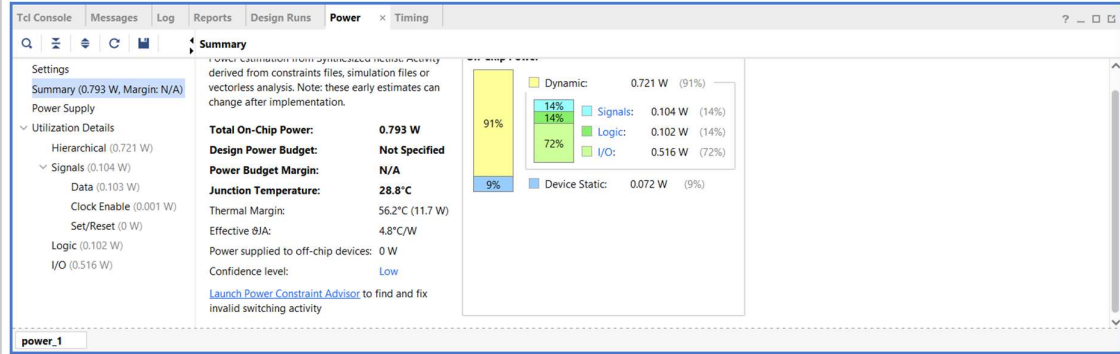
User Ignored Paths

> Unconstrained Paths

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): inf	Worst Hold Slack (WHS): inf	Worst Pulse Width Slack (WPWS): NA
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): NA
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: NA
Total Number of Endpoints: 67	Total Number of Endpoints: 67	Total Number of Endpoints: NA

There are no user specified timing constraints.

Timing Summary - timing_1

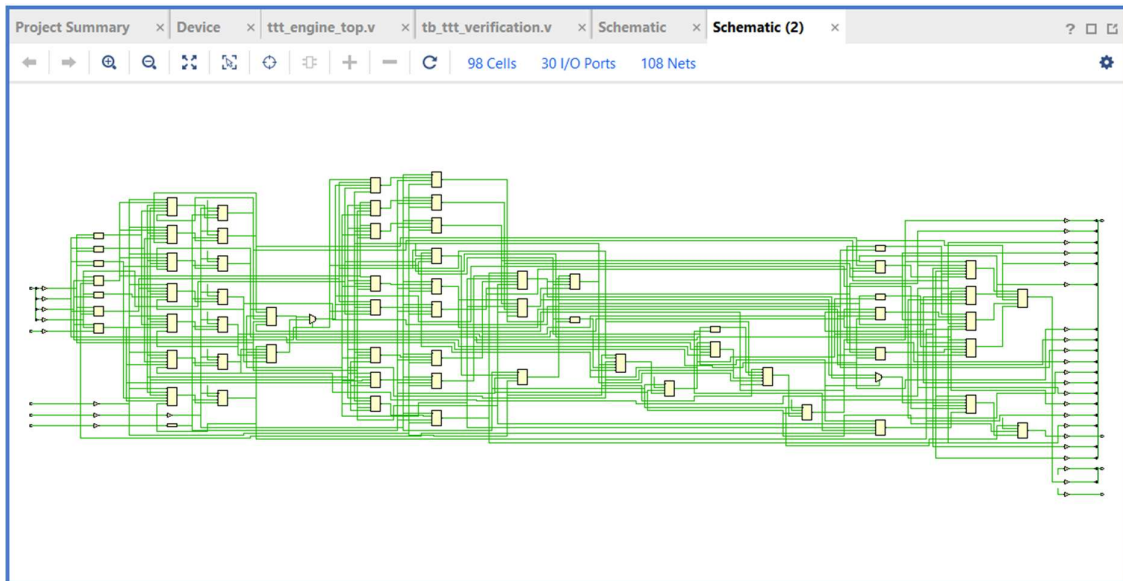


Tcl Console Messages Log Reports Design Runs Power Utilization x Timing

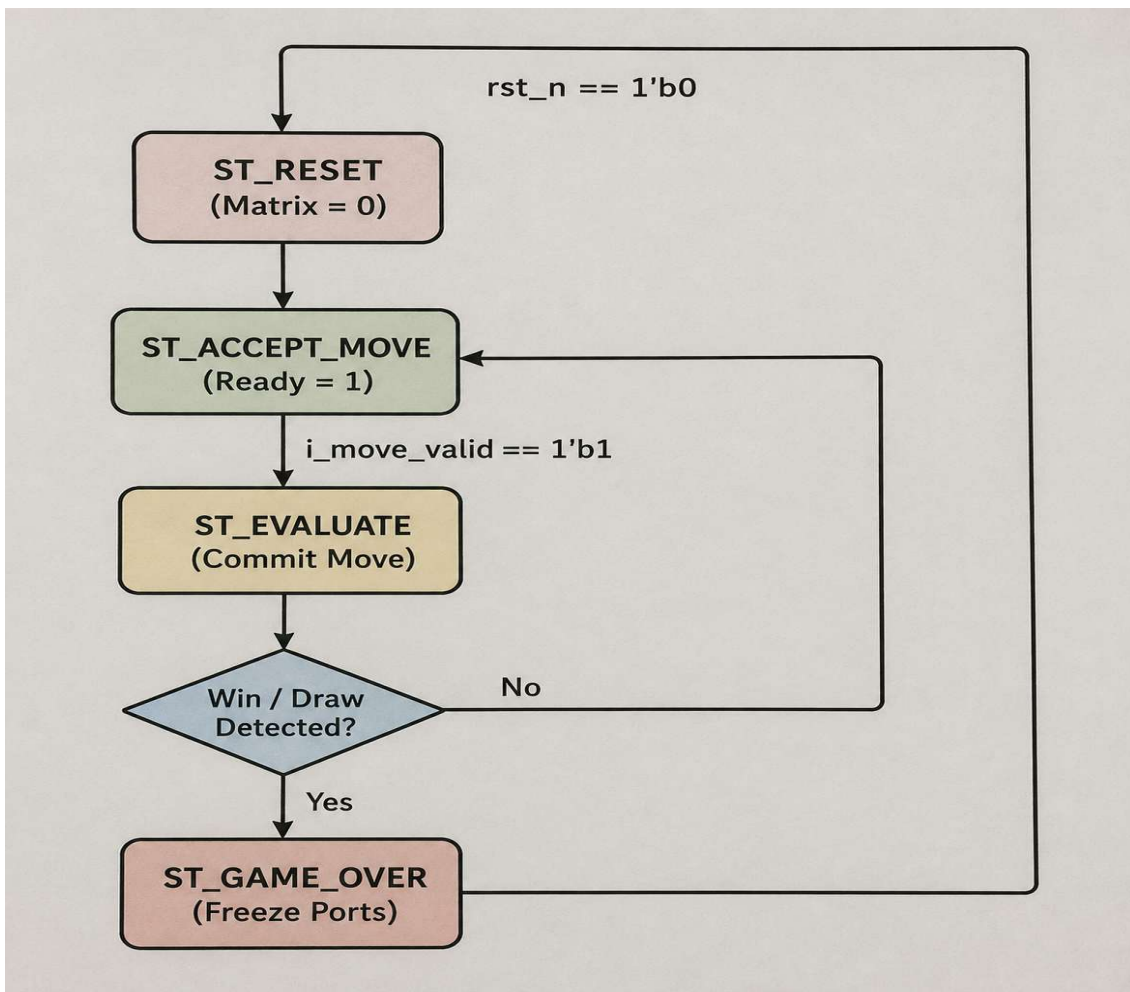
Hierarchy

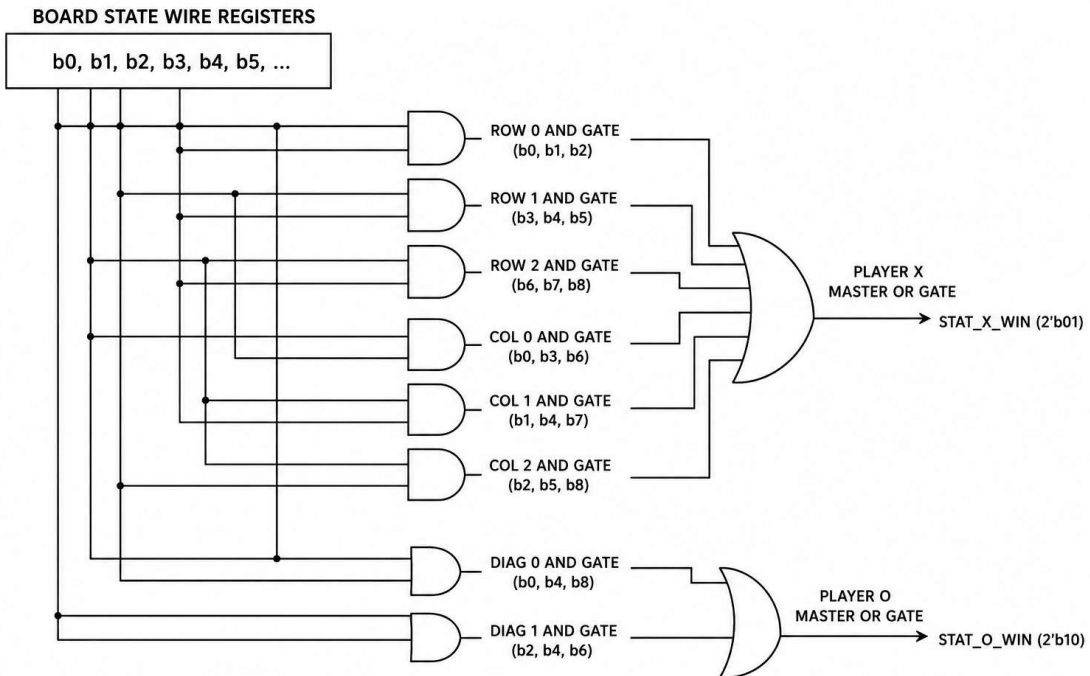
Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	Bonded IOB (210)	BUFGCTRL (32)
ttt_engine_top	41	19	2	30	1

utilization_1



Flow_diagrams

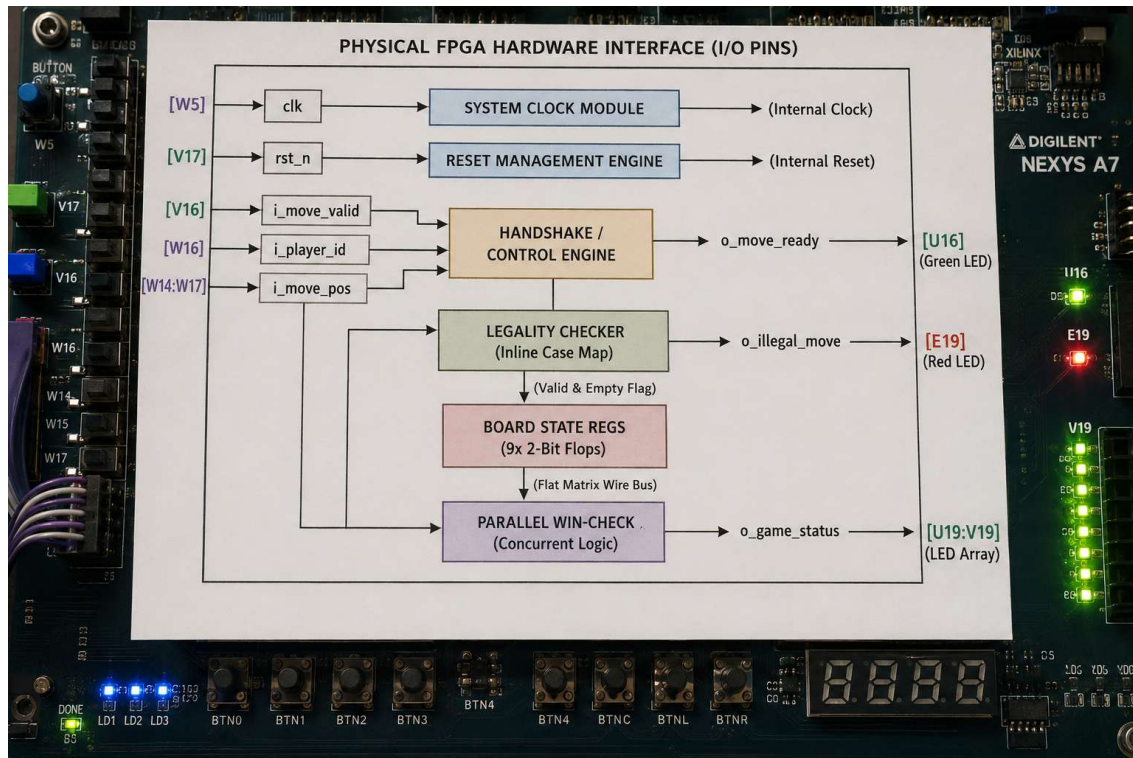




COL 0 [0,3,6]		COL 1 [1,4,7]		COL 2 [2,5,8]	
b0 [1:0] Top-Left Cell Position Switch: 0		b1 [1:0] Top-Center Cell Position Switch: 1		b2 [1:0] Top-Right Cell Position Switch: 2	
b3 [1:0] Center-Left Cell Position Switch: 3		b4 [1:0] Center Square Cell Position Switch: 4		b5 [1:0] Center-Right Cell Position Switch: 5	
b6 [1:0] Bottom-Left Cell Position Switch: 6		b7 [1:0] Bottom-Center Cell Position Switch: 7		b8 [1:0] Bottom-Right Cell Position Switch: 8	

ROW 1
[3,4,5]

ROW 2
[6,7,8]



Codes:

1. Core Logic Design (ttt_engine_top.v)

```
`timescale 1ns / 1ps
```

```
module ttt_engine_top (
    input wire clk,
    input wire rst_n,      // Synchronous active-low reset

    // Command Interface
    input wire i_move_valid, // Asserted when player move data is ready
    input wire [3:0] i_move_pos, // Target cell index (0 to 8)
    input wire i_player_id, // 0: Player X, 1: Player O
    output reg o_move_ready, // High when engine is ready to accept a move

    // Safety Diagnostic & Status Interfaces
    output reg o_illegal_move, // Pulled high for 1 cycle upon validation failure
    output reg [1:0] o_game_status, // 00: Active, 01: X Won, 10: O Won, 11: Draw
    output wire [17:0] o_board_flat // Flattened observability matrix for verification environments
);
```

```

// Encodings

localparam CELL_EMPTY   = 2'b00;

localparam CELL_PLAYER_X = 2'b01;

localparam CELL_PLAYER_O = 2'b10;


localparam STAT_ACTIVE  = 2'b00;

localparam STAT_X_WIN   = 2'b01;

localparam STAT_O_WIN   = 2'b10;

localparam STAT_DRAW    = 2'b11;


// Nine individual 2-bit board registers

reg [1:0] b0, b1, b2, b3, b4, b5, b6, b7, b8;

assign o_board_flat = {b8, b7, b6, b5, b4, b3, b2, b1, b0};


// Helper variable to fetch the target cell value synchronously

reg [1:0] target_cell;

always @(*) begin

    case (i_move_pos)

        4'd0: target_cell = b0; 4'd1: target_cell = b1; 4'd2: target_cell = b2;

        4'd3: target_cell = b3; 4'd4: target_cell = b4; 4'd5: target_cell = b5;

        4'd6: target_cell = b6; 4'd7: target_cell = b7; 4'd8: target_cell = b8;

        default: target_cell = 2'b11;

    endcase

end


// Fully Synchronous Game Engine Processing Pipeline

always @(posedge clk) begin

    if (!rst_n) begin

        b0 <= CELL_EMPTY; b1 <= CELL_EMPTY; b2 <= CELL_EMPTY;

        b3 <= CELL_EMPTY; b4 <= CELL_EMPTY; b5 <= CELL_EMPTY;

        b6 <= CELL_EMPTY; b7 <= CELL_EMPTY; b8 <= CELL_EMPTY;

        o_move_ready  <= 1'b1;

        o_illegal_move <= 1'b0;

    end else begin

        o_illegal_move <= 1'b0; // Default pulse configurations


        if (o_game_status == STAT_ACTIVE) begin

            o_move_ready <= 1'b1;

        end

    end

end

```

```

if (i_move_valid) begin
    // Move Validation Check
    if (i_move_pos < 4'd9 && target_cell == CELL_EMPTY) begin
        case (i_move_pos)
            4'd0: b0 <= (i_player_id == 1'b0) ? CELL_PLAYER_X : CELL_PLAYER_O;
            4'd1: b1 <= (i_player_id == 1'b0) ? CELL_PLAYER_X : CELL_PLAYER_O;
            4'd2: b2 <= (i_player_id == 1'b0) ? CELL_PLAYER_X : CELL_PLAYER_O;
            4'd3: b3 <= (i_player_id == 1'b0) ? CELL_PLAYER_X : CELL_PLAYER_O;
            4'd4: b4 <= (i_player_id == 1'b0) ? CELL_PLAYER_X : CELL_PLAYER_O;
            4'd5: b5 <= (i_player_id == 1'b0) ? CELL_PLAYER_X : CELL_PLAYER_O;
            4'd6: b6 <= (i_player_id == 1'b0) ? CELL_PLAYER_X : CELL_PLAYER_O;
            4'd7: b7 <= (i_player_id == 1'b0) ? CELL_PLAYER_X : CELL_PLAYER_O;
            4'd8: b8 <= (i_player_id == 1'b0) ? CELL_PLAYER_X : CELL_PLAYER_O;
        endcase
    end else begin
        o_illegal_move <= 1'b1; // Trigger fault block
    end
end

end else begin
    o_move_ready <= 1'b0;
end

end

end

// Fully Synchronous Parallel Win-Checker
always @(posedge clk) begin
    if (!rst_n) begin
        o_game_status <= STAT_ACTIVE;
    end else if (o_game_status == STAT_ACTIVE) begin
        // 1. Rows Check
        if ((b0 == CELL_PLAYER_X && b1 == CELL_PLAYER_X && b2 == CELL_PLAYER_X) ||
            (b3 == CELL_PLAYER_X && b4 == CELL_PLAYER_X && b5 == CELL_PLAYER_X) ||
            (b6 == CELL_PLAYER_X && b7 == CELL_PLAYER_X && b8 == CELL_PLAYER_X)) o_game_status <= STAT_X_WIN;

        else if ((b0 == CELL_PLAYER_O && b1 == CELL_PLAYER_O && b2 == CELL_PLAYER_O) ||
            (b3 == CELL_PLAYER_O && b4 == CELL_PLAYER_O && b5 == CELL_PLAYER_O) ||
            (b6 == CELL_PLAYER_O && b7 == CELL_PLAYER_O && b8 == CELL_PLAYER_O)) o_game_status <= STAT_O_WIN;
    end
end

```

```

// 2. Columns Check
else if ((b0 == CELL_PLAYER_X && b3 == CELL_PLAYER_X && b6 == CELL_PLAYER_X) ||
         (b1 == CELL_PLAYER_X && b4 == CELL_PLAYER_X && b7 == CELL_PLAYER_X) ||
         (b2 == CELL_PLAYER_X && b5 == CELL_PLAYER_X && b8 == CELL_PLAYER_X)) o_game_status <= STAT_X_WIN;

else if ((b0 == CELL_PLAYER_O && b3 == CELL_PLAYER_O && b6 == CELL_PLAYER_O) ||
         (b1 == CELL_PLAYER_O && b4 == CELL_PLAYER_O && b7 == CELL_PLAYER_O) ||
         (b2 == CELL_PLAYER_O && b5 == CELL_PLAYER_O && b8 == CELL_PLAYER_O)) o_game_status <= STAT_O_WIN;

// 3. Diagonals Check
else if ((b0 == CELL_PLAYER_X && b4 == CELL_PLAYER_X && b8 == CELL_PLAYER_X) ||
         (b2 == CELL_PLAYER_X && b4 == CELL_PLAYER_X && b6 == CELL_PLAYER_X)) o_game_status <= STAT_X_WIN;

else if ((b0 == CELL_PLAYER_O && b4 == CELL_PLAYER_O && b8 == CELL_PLAYER_O) ||
         (b2 == CELL_PLAYER_O && b4 == CELL_PLAYER_O && b6 == CELL_PLAYER_O)) o_game_status <= STAT_O_WIN;

// 4. Draw Check
else if (b0 != CELL_EMPTY && b1 != CELL_EMPTY && b2 != CELL_EMPTY &&
         b3 != CELL_EMPTY && b4 != CELL_EMPTY && b5 != CELL_EMPTY &&
         b6 != CELL_EMPTY && b7 != CELL_EMPTY && b8 != CELL_EMPTY) begin
    o_game_status <= STAT_DRAW;
end

end

end

end

endmodule

```

2. Verification Testbench (tb_ttt_verification.v)

```
`timescale 1ns / 1ps
```

```
module tb_ttt_verification();
```

```
    reg clk;
```

```
    reg rst_n;
```

```
    reg i_move_valid;
```

```
    reg [3:0] i_move_pos;
```

```
    reg i_player_id;
```

```
    wire o_move_ready;
```

```

wire o_illegal_move;

wire [1:0] o_game_status;

wire [17:0] o_board_flat;


localparam STAT_ACTIVE = 2'b00;

localparam STAT_X_WIN = 2'b01;

localparam STAT_O_WIN = 2'b10;

localparam STAT_DRAW = 2'b11;


// Instantiate Unit Under Test (UUT)
ttt_engine_top uut (

    .clk(clk), .rst_n(rst_n),

    .i_move_valid(i_move_valid), .i_move_pos(i_move_pos), .i_player_id(i_player_id),

    .o_move_ready(o_move_ready), .o_illegal_move(o_illegal_move),

    .o_game_status(o_game_status), .o_board_flat(o_board_flat)

);


// 100MHz System Clock Generation (10ns period)

always #5 clk = ~clk;


initial begin

    // Reset Phase

    clk = 0; rst_n = 0; i_move_valid = 0; i_move_pos = 0; i_player_id = 0;

    #35;

    rst_n = 1; // Release reset cleanly

    #15;

    $display("===== STARTING RTL ENGINE VERIFICATION MATRIX =====");


    // --- TEST CASE 1: Validation Fault & Illegal Collision Check ---

    $display("[TC1] Driving standard operational input to Center Square (4)");

    @(negedge clk); // Drive safely on the falling edge to prevent race hazards

    i_move_pos <= 4; i_player_id <= 0; i_move_valid <= 1;


    @(negedge clk);

    i_move_valid <= 0;


    $display("[TC1] Attempting hostile collision intercept on Center Square (4)");

```

```

@(negedge clk);

i_move_pos <= 4; i_player_id <= 1; i_move_valid <= 1;

@(negedge clk);

i_move_valid <= 0;

// Wait 2 cycles for the internal registers to log the diagnostic flag
@(posedge clk);

#1;

if (o_illegal_move === 1'b1)
    $display("[TC1 SUCCESS] Engine rejected collision step successfully.");
else
    $display("[TC1 FAILURE] Error: Engine register safety check failed!");

// --- TEST CASE 2: Structural Diagonal Completion Win Vector ---
$display("[TC2] Initiating parallel path game matrix checks...");

// Move 1: O plays top-left
@(negedge clk); i_move_pos <= 0; i_player_id <= 1; i_move_valid <= 1;
@(negedge clk); i_move_valid <= 0;

// Move 2: X plays top-center
@(negedge clk); i_move_pos <= 1; i_player_id <= 0; i_move_valid <= 1;
@(negedge clk); i_move_valid <= 0;

// Move 3: O plays bottom-right
@(negedge clk); i_move_pos <= 8; i_player_id <= 1; i_move_valid <= 1;
@(negedge clk); i_move_valid <= 0;

// Move 4: X plays top-right
@(negedge clk); i_move_pos <= 2; i_player_id <= 0; i_move_valid <= 1;
@(negedge clk); i_move_valid <= 0;

// Move 5: O plays center -> Wins diagonally!
@(negedge clk); i_move_pos <= 4; i_player_id <= 1; i_move_valid <= 1;
@(negedge clk); i_move_valid <= 0;

// Allow status pipeline register to update on next rising edge

```

```
@(posedge clk);

#5;

if (o_game_status === STAT_O_WIN)
    $display("[TC2 SUCCESS] Engine recognized combinational diagonal winner resolution.");
else
    $display("[TC2 FAILURE] Error: Engine failed parallel vector match calculations.");

$display("=====  
RTL VERIFICATION SUITE COMPLETE  
=====");

$finish;

end

endmodule
```